

정우태

Unity Client Developer

물리 엔진과 구조 개조 경험을 가진 Unity 2D/3D 클라이언트 개발자

문제를 해결하고 구조를 정리하고 성능을 높인
정확한 디버깅하는 능력에 강점이 있습니다

Release

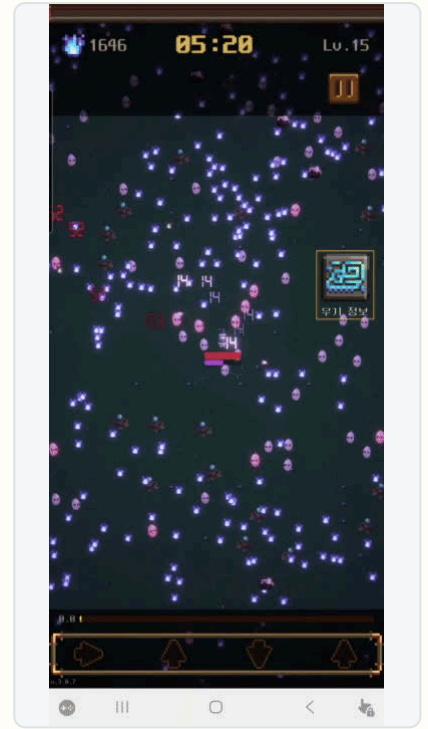
Google Play,
itch.io

Systems

DOTS, EventBus,
XR, UI

Workflow

AI guardrail, tests,
QA



어떤 개발자인가

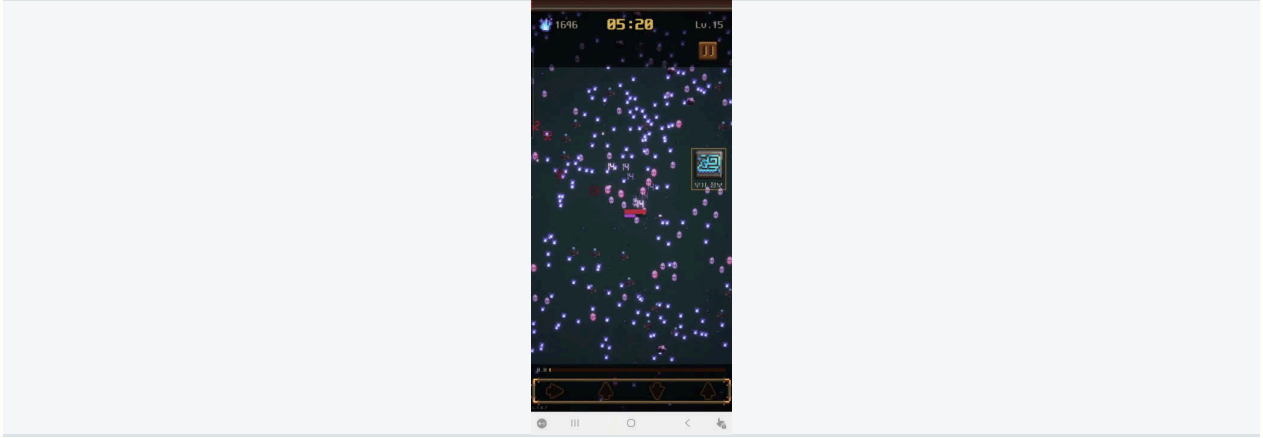
저는 기능을 많이 붙이는 것보다, 기능이 계속 붙을 수 있는 구조를 만드는 쪽에 관심이 많습니다. 작은 프로젝트에서도 배포, 최적화, 입력, 저장, 광고, UI, 테스트가 한번에 얹히면 유지보수 비용이 빠르게 늘어난다는 것을 직접 겪었습니다.

그래서 프로젝트를 정리할 때는 먼저 실행 가능한 기준본을 남기고, 병목과 책임 범위를 나눈 뒤, 새 구조가 실제로 더 나은지 수치나 테스트로 확인하려고 합니다. Ghost March에서는 출시된 Legacy 구조를 보존한 채 DOTS 구조를 실험했고, Match3LogueLike에서는 AI가 임의로 참조 방향을 깨지 않도록 작업 규칙과 테스트를 먼저 세웠습니다.

면접이나 협업 상황에서도 같은 방식을 선호합니다. 문제를 바로 해결하려 하기보다, 현재 무엇이 확실하고 무엇이 추정인지 분리합니다. 그 다음 작게 재현하고, 원인을 좁히고, 팀원이 이어받을 수 있는 형태로 남깁니다.

핵심 키워드: 출시 경험, 성능 재설계, 의존성 경계, 입력/렌더/UI 트러블슈팅, AI 협업 구조화

출시작을 기본본으로 남긴 뒤 다시 설계했습니다



Ghost March는 공모전 수상 후 Google Play에 출시한 모바일 액션 게임입니다. 출시 이후에는 단순히 코드를 갈아엎기보다, 기존 MonoBehaviour 구조를 기본본으로 보존하고 새 구조를 비교할 수 있는 상태를 먼저 만들었습니다.

초기 구조에서는 `GameManager.instance`가 상태, 입력, UI, 저장을 폭넓게 소유했고, `Weapon.Update()`의 switch 기반 분기는 무기가 늘어날수록 확장 비용을 키웠습니다. 대량 적, 데미지 텍스트, UI Toolkit 갱신도 프레임 비용을 키우는 원인이었습니다.

재설계는 Enemy, Bullet, Drop, Wave, Damage Event를 DOTS 수직 슬라이스로 분리하는 방식으로 진행했습니다. MonoBehaviour는 UI, 입력, 씬 부트스트랩, 외부 서비스 연동으로 제한했습니다.

9 fps

Legacy 1500 enemies

67 fps

DOTS 4096 enemies

109.5ms ->

14.92ms

CPU frame time

문제는 기능 단위가 아니라 원인 단위로 쪼갬니다

GPU Instancing	전환 후 플레이어/적/소울이 사라지는 현상을 scene value와 fallback 부재로 나눠 확인했습니다.
Enemy Separation	더블버퍼가 chase 이동을 덮어쓰던 문제를 <code>PositionDelta</code> 로 분리해 해결했습니다.
UI Toolkit	매 프레임 <code>label.text</code> 갱신을 변경 감지 방식으로 줄여 CPU/GPU 비용을 낮췄습니다.
Android Release	AAB, versionCode, manifest, Billing metadata, R8 keep rule, Play Core 의존성 경계를 검증했습니다.

691 -> 4

instanced batches

16.93 ->**11.22ms**

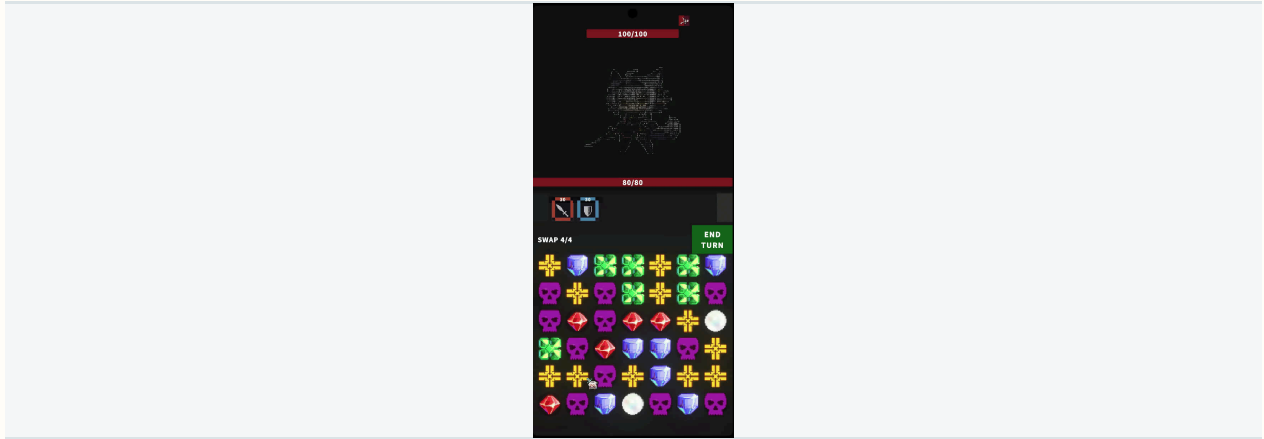
UI Toolkit GPU

17.47 ->**11.43ms**

UI Toolkit CPU

이 페이지는 프로젝트 소개보다, 문제를 어떤 단위로 바라보고 해결했는지를 보여주는 장으로 구성했습니다.

AI를 쓰기 전에 AI가 건드릴 수 없는 경계를 정했습니다



Match3LogueLike는 모바일 매치3 로그라이크 구조 설계 프로젝트입니다. 여기서 강조하고 싶은 지점은 AI 사용 자체가 아니라, AI가 구조를 망가뜨리지 않도록 먼저 규칙을 만든 방식입니다.

Data, Core, Board, Combat, Roguelike, Meta, UI, Ads의 방향을 나누고, Board와 Combat의 직접 참조를 금지했습니다. 통신은 EventBus로 제한했고, Data 영역은 MonoBehaviour 없이 ScriptableObject, interface, enum 중심으로 유지했습니다.

작업 방식도 Coordinator, Worker, Verify 역할로 나누었습니다. Data MonoBehaviour 금지, Find 금지, Update 할당 금지 hook을 적용했고, merge 전 PlayMode 테스트와 실제 화면 QA를 함께 확인했습니다.

34

C# test files

10/10

RewardOverlay PlayMode

4/4

SaveManager EditMode

팀 프로젝트와 공개 개발에서 확인한 것



IronSarcophagus는 2인 팀 VR 전차 협동 게임입니다. 네트워크 기반 구조, 포수 조준, 탄도 계산, 적 AI를 담당했습니다. 총구 기준 5개 포인트 Linecast로 노출도를 계산하고, NetworkVariable 기반으로 부위별 HP, 장갑, 관통을 처리했습니다.



LostCone은 공개 개발과 플레이테스트를 통해 방향을 바꾼 팬게임입니다. 터치스크린 노트북의 Control Scheme auto-switch 때문에 키보드 입력이 간헐적으로 중단되는 문제를 추적하고, scheme 강제와 prefab 기본값 수정으로 해결했습니다.

작게 완성하고, 다음 구조로 이어갑니다

Project	역할	포트폴리오에서 보여줄 점
Cubika	Solo / Release	WebGL 배포, 큐브 회전/낙하/밀기, 3초 game-over grace, 모바일 조작과 광고/IAP scaffold 확장.
Dong_Mae	Team / Implementation	타일맵, 문, 로프, 점프패드, 함정, 보스 패턴, 카메라/페이드 연출 구현.
MyTokenMate	AI Workflow Tool	Claude, Codex, Gemini, Copilot 로컬 로그 기반 사용량 overlay. 독립 프로젝트보다 작업 방식 보조 근거로 짧게 언급.

- GitHub
- Ghost March
- Cubika
- Email